GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-22

今日榜单

1 **php/php-src** C
The PHP Interpreter NEW

2 **anthropics/knowledge-work-plugins** Python
Open source repository of plugins primarily intended for knowledge workers to us... NEW

3 **DietrichGebert/ponytail** JavaScript
Makes your AI agent think like the laziest senior dev in the room. The best code... 3天

4 **cloudflare/security-audit-skill** JavaScript
A coding-agent skill for multi-phase security audits with independently verified... 3天

5 **Graphify-Labs/graphify** Python
Turn any codebase, with its docs, SQL schemas, configs, and PDFs, into a queryab... NEW

6 **supabase/supabase** TypeScript
The Postgres development platform. Supabase gives you a dedicated Postgres datab... 2天

7 **fastapi/fastapi** Python
FastAPI framework, high performance, easy to learn, fast to code, ready for prod... NEW

8 **luongnv89/claude-howto** Python
A visual, example-driven guide to Claude Code — from basic concepts to advanced ... NEW

9 **Meloong-Git/PCL** Visual Basic .NET
Minecraft 启动器 Plain Craft Launcher（PCL）。 NEW

#1 C NEW

php-src

The PHP Interpreter

Stars **40,901** Forks **8,281** Today **+110**

https://github.com/php/php-src

项目简介：php/php-src 是 PHP 官方解释器的源代码仓库，也就是 PHP 语言本身的本体实现。它解决的问题是为全球 Web 开发提供一个快速、灵活、务实的通用脚本语言运行时，从个人博客到大型网站都依赖它运行。

核心功能： - PHP 语言解释执行引擎（Zend 引擎），负责词法/语法解析、编译为 opcode 并执行。 - 内置大量标准扩展（如 SPL、PDO、JSON、mbstring 等），开箱即用地覆盖常见开发需求。 - 完善的测试套件与 `make test` 机制，支撑语言级别的质量保障。 - 扩展机制与 PIE/PECL 生态，允许第三方以 C 扩展形式扩展语言能力。

适用场景：主要面向 PHP 语言核心开发者、扩展开发者，以及需要从源码编译定制 PHP（如开启调试、裁剪扩展、适配特定平台）的运维和平台工程师；普通业务开发者通常使用预编译包而非直接接触此仓库。

技术亮点：核心用 C 实现，采用 Zend 虚拟机架构，性能与可移植性兼顾；构建流程基于 autoconf/bison/re2c，工具链成熟；集成 OSS-Fuzz 持续模糊测试，安全性与稳定性投入可见；贡献流程规范，新特性需走 RFC 与投票机制，工程治理成熟。

上手建议：先按 README 安装依赖并完成 `./buildconf && ./configure --enable-debug && make -j` 的最小构建，跑通 `make test` 熟悉测试体系；随后从修复带 GitHub issue 的小 bug 入手提交 PR，若想新增特性则先阅读 wiki.php.net 上的 RFC 流程再动手。

#2 Python NEW

knowledge-work-plugins

Open source repository of plugins primarily intended for knowledge workers to use in Claude Cowork

Stars **31,364** Forks **3,517** Today **+303**

https://github.com/anthropics/knowledge-work-plugins

项目简介：这是 Anthropic 官方开源的知识工作插件库，为 Claude Cowork（兼容 Claude Code）提供面向特定岗位的"专业化配置包"。它解决的核心问题是：让 Claude 不只是通用助手，而是能按你的团队工具、术语和流程来工作，产出更一致、更专业的结果。

核心功能：一是 11 个开箱即用的岗位插件（销售、客服、产品、市场、法务、财务、数据分析、企业搜索、生物研究、生产力、插件管理等），每个都捆绑了对应职能的技能、连接器和斜杠命令；二是通过 MCP 连接器打通 Slack、Notion、Jira、HubSpot、Snowflake 等外部工具，让 Claude 能直接读取和操作真实业务数据；三是技能（Skills）自动触发 + 斜杠命令显式调用（如 `/sales:call-prep`）的双层交互模式；四是提供 `cowork-plugin-management` 插件，支持团队自行创建和定制插件。

适用场景：主要面向各类知识工作者——销售做客户调研和通话准备、客服做工单分流和回复草稿、PM 写规格和路线图、法务审合同、财务做对账和报表、数据分析师写 SQL 和建看板等。最典型的用法是团队基于官方模板，把公司自己的工具链、术语和流程注入进去，形成"为你们团队定制"的 Claude。

技术亮点：插件采用统一的文件化结构（`plugin.json` 清单 + `.mcp.json` 工具连接 + `commands/` + `skills/`），完全基于文件、易于版本管理和分发；通过 MCP（Model Context Protocol）标准化对接外部系统，连接器生态可扩展；"技能自动触发 + 命令显式调用"的设计让领域知识既能被动生效也能主动调用，兼顾了易用性和可控性。

上手建议：普通用户最直接的方式是在 Cowork 里从 claude.com/plugins 安装，或在 Claude Code 中用 `claude plugin marketplace add anthropics/knowledge-work-plugins` 后安装具体插件（如 `sales@knowledge-work-plugins`）。想深入定制或贡献的话，建议先精读某个插件（如 productivity）的目录结构和 `plugin.json`，理解 skills/commands 的写法，再用 `cowork-plugin-management` 插件动手改造，最后按官方结构提交 PR。

#3 JavaScript 连续 3 天

ponytail

Makes your AI agent think like the laziest senior dev in the room. The best code is the code you never wrote.

Stars **172,822** Forks **9,407** Today **+1,949**

https://github.com/DietrichGebert/ponytail

项目简介：Ponytail 是一个给 AI 编程助手（Agent）加装的"技能包"，让 AI 在写代码时模仿那种沉默寡言、只写一行的资深老工程师——只写任务真正需要的东西，绝不堆砌冗余代码。它解决的是 AI 编码助手常见的"过度生成"问题：代码越写越多、token 越烧越贵、时间越拖越长。

核心功能：一是"极简输出"约束，通过技能提示让 Agent 大幅削减代码量（官方基准称代码量降 53%、token 降 45%）；二是保留质量底线，明确要求不砍校验、错误处理、安全与可访问性；三是跨 Agent 兼容，宣称支持 20 种 Agent（如 Claude Code）；四是内置基准测试与盲评结果，用数据证明"更少代码、更安全"。

适用场景：适合日常用 Claude Code 等 AI 编程工具做真实项目（如 FastAPI + React 仓库）的开发者，尤其是厌倦了 AI 生成大段样板代码、想降低 token 成本与审查负担的团队；也适合关注 AI 编码质量与安全性的研究者。

技术亮点：核心不是模型微调，而是通过"技能/skill"层面的提示工程与规则约束来改变 Agent 行为；项目用 39 个任务、多次重复运行的对照基准来量化效果，并特别强调在 SQL 注入、路径穿越、伪造 token 等 6 类安全任务上全部通过，说明"精简"没有牺牲安全性，这是它区别于普通 prompt 技巧的关键。

上手建议：从 README 的安装说明入手，通过 npm 包（@dietrichgebert/ponytail）接入你常用的 Agent，先在一个小项目上对比开启前后的代码量与测试覆盖；想深入可阅读 benchmarks/results 下的评测方法，贡献方向可关注多 Agent 适配、基准任务扩充与安全用例补充。

#4 JavaScript 连续 3 天

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **38,457** Forks **2,394** Today **+2,979**

https://github.com/cloudflare/security-audit-skill

项目简介：这是 Cloudflare 开源的一个「编码 Agent 技能包」，本质是一套结构化的提示词与校验脚本集合，让 Claude Code、Cursor 等编码 Agent 变身为多阶段安全审计员。它解决的核心问题是：让 AI 驱动的漏洞挖掘从"凭感觉扫一遍"变成有覆盖台账、可独立复核、结果机器可读的工程化流程。

核心功能：① 六阶段流水线——侦察建图 → 按覆盖台账分配猎人 → 候选漏洞交叉验证 → 结构化输出 → 独立复核 → 生成报告；② 用 `coverage-ledger.json` 追踪审计覆盖面，避免漏审与重复；③ 每个候选漏洞交给全新的"反方验证者"尝试证伪，结论分为 confirmed / needs_validation / rejected 三档；④ 两个零依赖校验脚本（`validate-findings.cjs`、`validate-coverage-ledger.cjs`）保证输出符合 JSON Schema；⑤ 按攻击面拆分的十余份提示词文档（内存安全、LLM 注入、Web 协议、供应链、云部署等），可按目标类型选用。

适用场景：安全工程师或研发团队在发版前对自有代码库做自动化审计；也适合安全研究者把它当作"自建漏洞挖掘 harness"的起点——README 明确说这就是 Cloudflare 内部大规模漏洞发现系统的单仓库雏形。对同一仓库反复运行是增量式的，适合持续审计。

技术亮点：设计上最值得关注的是"对抗式验证"思路——不让发现者自证，而是强制用独立 Agent 证伪，并区分"已确认"与"待验证"的严重性语义（needs_validation 不标 severity）；其次是"覆盖台账"作为一等公民，把审计完整性量化并可校验；再者，整个项目几乎没有运行时代码，主体是精心编排的提示词工程加两个纯校验器，体现了"Agent 技能即协议"的新范式。

上手建议：用 `npx skills add` 一条命令安装，然后在目标仓库里对 Agent 说"security audit this codebase"即可跑通。想深入的话，先读 `SKILL.md` 理解原则与反模式，再按目标类型挑对应的攻击类文档；贡献方向可以是补充新的攻击类提示词、完善 schema 校验规则，或为验证器补测试用例。注意它是提示词驱动，效果高度依赖所用 Agent 模型的能力。

#5 Python NEW

graphify

Turn any codebase, with its docs, SQL schemas, configs, and PDFs, into a queryable knowledge graph. A /graphify skill for Claude Code, Cursor, Codex, and Gemini CLI: local deterministic AST parsing, every edge explained, no vector store.

Stars **132,475** Forks **12,695** Today **+1,407**

https://github.com/Graphify-Labs/graphify

项目简介：Graphify 是一个把代码库、文档、SQL schema、配置和 PDF 等异构资料统一转成**可查询知识图谱**的工具。它解决的是 AI 编程助手在大型项目中"靠 grep 翻文件"效率低、上下文碎片化的问题，让 agent 用图查询替代全文检索。

核心功能： - 输入 `/graphify` 命令即可为整个项目建图，覆盖代码、文档、PDF、图片、视频等多模态内容。 - 代码部分用 tree-sitter AST 做**本地确定性解析**，不调用 LLM、数据不出本机。 - 每条边都带来源标注：EXTRACTED（源文件中显式存在）或 INFERRED（由 Graphify 推断），可追溯可信度。 - 不依赖向量库，直接以图结构做查询，替代传统 RAG 的 embedding 检索路径。 - 作为 Claude Code、Cursor、Codex、Gemini CLI 的 skill 插件形态集成。

适用场景：适合在**中大型代码库**上工作的开发者与 AI agent——需要跨文件理解调用关系、数据流向、文档与代码对应关系时；也适合需要把 SQL schema、配置、PDF 规范一并纳入上下文的企业工程团队。小型脚本项目收益有限。

技术亮点：最值得关注的是"**确定性 AST 解析 + 显式/推断边分离**"的设计——代码结构不经过 LLM，保证可复现和隐私；语义补充（文档、PDF）才走模型。放弃向量库、改用知识图谱做检索，是对"RAG 是否万能"的一次明确表态，边来源标注则直接回应了 LLM 生成关系不可信的老问题。

上手建议：先在目标项目的 AI 助手里输入 `/graphify` 跑一遍，观察生成的边标注质量；再对照 `docs.graphify.com` 的文档理解查询语法。想贡献的话，从 tree-sitter 支持的语言扩展和边的推断规则入手门槛较低，CI 与 PyPI 包（`graphifyy`）结构清晰，便于本地调试。

注：132k 星标与 YC S26 背景下的高速增长值得留意，建议实际验证图谱质量后再决定是否深入依赖。

#6 TypeScript 连续 2 天

supabase

The Postgres development platform. Supabase gives you a dedicated Postgres database to build your web, mobile, and AI applications.

Stars **113,322** Forks **16,991** Today **+683**

https://github.com/supabase/supabase

项目简介：Supabase 是一个基于 PostgreSQL 的开源后端开发平台，目标是用企业级开源工具复刻 Firebase 的核心能力，让开发者无需自建后端即可快速搭建 Web、移动端和 AI 应用。它解决的是"想要 Firebase 式开发体验，但又不想被闭源云服务锁定"这一核心矛盾。

核心功能：提供托管/自托管的 Postgres 数据库；内置认证授权（Auth）；根据数据库结构自动生成 REST、GraphQL 和实时订阅 API；支持数据库函数与 Edge Functions；提供文件存储以及 AI 向量/Embeddings 工具包，并配有可视化管理 Dashboard。

适用场景：适合需要快速验证产品的独立开发者和小团队、想要迁移出 Firebase 的项目、以及希望数据自主可控的企业。典型场景包括 SaaS 后端、移动 App 后端、实时协作应用，以及需要向量检索的 AI 应用。

技术亮点：架构上采用"组合优秀开源件"的策略——PostgREST 把数据库直接变成 REST API、Realtime 用 Elixir 监听 Postgres 复制流并通过 WebSocket 广播、GoTrue 负责 JWT 认证、pg_graphql 提供 GraphQL 扩展，各组件模块化且可独立替换。这种设计让数据库成为唯一真相源，权限也统一由 Postgres 的 RLS 策略管理。

上手建议：最省事的方式是直接在 supabase.com 注册托管版，零安装即可体验；想本地开发或自托管，可参考官方 local-development 与 hosting 文档。贡献代码前先读 DEVELOPERS.md，日常问题优先走 GitHub Discussions 和 Discord 社区。

#7 Python NEW

fastapi

FastAPI framework, high performance, easy to learn, fast to code, ready for production

Stars **103,511** Forks **10,076** Today **+96**

https://github.com/fastapi/fastapi

项目简介：FastAPI 是一个基于 Python 类型提示（type hints）构建的现代高性能 Web 框架，专为开发 API 而设计。它解决了传统 Python Web 框架在开发效率、运行时性能和文档自动化之间难以兼顾的问题，让开发者用更少的代码获得生产级 API 服务。

核心功能：一是基于 Python 类型注解自动完成请求参数校验与序列化；二是自动生成符合 OpenAPI/JSON Schema 标准的交互式 API 文档（Swagger UI 与 ReDoc）；三是原生支持异步（async/await），性能接近 NodeJS 和 Go；四是依赖注入系统，便于复用鉴权、数据库连接等逻辑；五是编辑器自动补全友好，显著减少调试时间。

适用场景：适合需要快速构建 RESTful API 的后端开发者，尤其是数据科学/机器学习模型服务化（把模型包装成 API）、微服务、内部工具后端等场景。对追求开发速度与文档自动化的中小团队、以及从 Flask/Django 迁移到异步架构的项目尤为合适。

技术亮点：底层由 Starlette（ASGI 异步 Web 框架）和 Pydantic（数据校验）驱动，二者分别负责高性能网络层和类型安全的数据层，这是其性能与开发体验兼得的关键。将类型提示同时用于运行时校验、序列化和文档生成，实现了"一次声明、多处复用"的设计哲学。

上手建议：从官方文档的 Tutorial 逐步学习（中文文档完善），先跑通一个最小示例再深入依赖注入与安全模块。贡献方面，可先阅读贡献指南，从文档改进、issue 复现或小型 bug 修复入手，项目测试覆盖率高、CI 流程规范，适合作为参与大型开源项目的起点。

#8 Python NEW

claude-howto

A visual, example-driven guide to Claude Code — from basic concepts to advanced agents, with copy-paste templates that bring immediate value.

Stars **42,760** Forks **5,360** Today **+428**

https://github.com/luongnv89/claude-howto

项目简介：这是一个面向 Claude Code 的可视化、示例驱动的学习指南，把官方文档里零散的功能描述重新组织成一条从入门到进阶的完整路径。它要解决的核心痛点是：装好 Claude Code、跑过几个 prompt 之后，用户往往不知道如何把斜杠命令、hooks、记忆、子代理、MCP 等能力组合成真正省时的工作流。

核心功能：一是 10 个渐进式教程模块，覆盖从斜杠命令到自定义代理团队的几乎所有 Claude Code 特性；二是大量可直接复制粘贴的配置模板，包括 CLAUDE.md、hook 脚本、MCP 配置、子代理定义和插件包；三是内置自评与测验机制（`/self-assessment`、`/lesson-quiz`），能在 Claude Code 内直接定位知识缺口并生成个性化路线。

适用场景：适合已经安装 Claude Code 但停留在"会打字不会编排"阶段的开发者，也适合想把 Claude Code 引入团队工作流、需要现成模板参考的工程团队。典型场景包括搭建自动化代码审查流水线、部署流程、文档生成等需要多特性协同的任务。

技术亮点：用 Mermaid 图解释每个特性的内部机制，强调"知其所以然"而非只给操作步骤；把学习路径本身做成了可在 Claude Code 内运行的交互式工具（自评、测验命令），让文档与工具边界模糊化。项目以 MIT 协议开源，并提供多语言 README，社区传播性较强。

上手建议：先在 Claude Code 中运行 `/self-assessment` 确定自己的水平，再按 `LEARNING-ROADMAP.md` 的顺序推进，边学边把模板复制进自己的项目。想贡献的话，可从补充模板案例、修正文档或翻译多语言版本入手，注意项目当前星标增长很快，提交前先看 issue 和贡献指南避免重复劳动。

#9 Visual Basic .NET NEW

PCL

Minecraft 启动器 Plain Craft Launcher（PCL）。

Stars **8,598** Forks **1,187** Today **+115**

https://github.com/Meloong-Git/PCL

项目简介：PCL（Plain Craft Launcher）是一款面向 Minecraft 玩家的第三方启动器，用 Visual Basic .NET 开发，旨在提供比官方启动器更轻量、功能更丰富的游戏启动与管理体验。项目开源了大部分源代码，涵盖 UI 库、动画、下载和启动等核心模块。

核心功能：一是 Minecraft 多版本管理与一键启动；二是内置资源下载模块，支持游戏本体、模组、整合包等的获取；三是自研 UI 库与动画系统，提供流畅的界面交互；四是内置帮助文档与功能投票机制，方便用户自助解决问题并参与产品方向决策。

适用场景：主要面向中文 Minecraft 玩家，尤其是需要管理多个游戏版本、安装模组或整合包的用户；也适合希望参考 .NET 桌面应用 UI 与动画实现方式的开发者学习。

技术亮点：用 VB.NET 实现了一套自绘 UI 库和动画模块，在 WinForms 体系下做出了较现代的界面效果；代码按发布节奏同步开源，模块划分清晰（UI、下载、启动分离），对研究桌面启动器架构有参考价值。

上手建议：普通用户直接从官网下载正式版使用即可；想参与贡献或二次开发，建议先阅读仓库中的各模块源码，从 Issues 和 Discussions 入手了解待办方向，帮助文档库（PCL2Help）也是低门槛的贡献入口。注意代码非实时同步，需以正式版发布为准。

数据来源：github.com/trending

报告日期：2026-10-22

由 DeepSeek AI 提供智能分析 | 自动生成